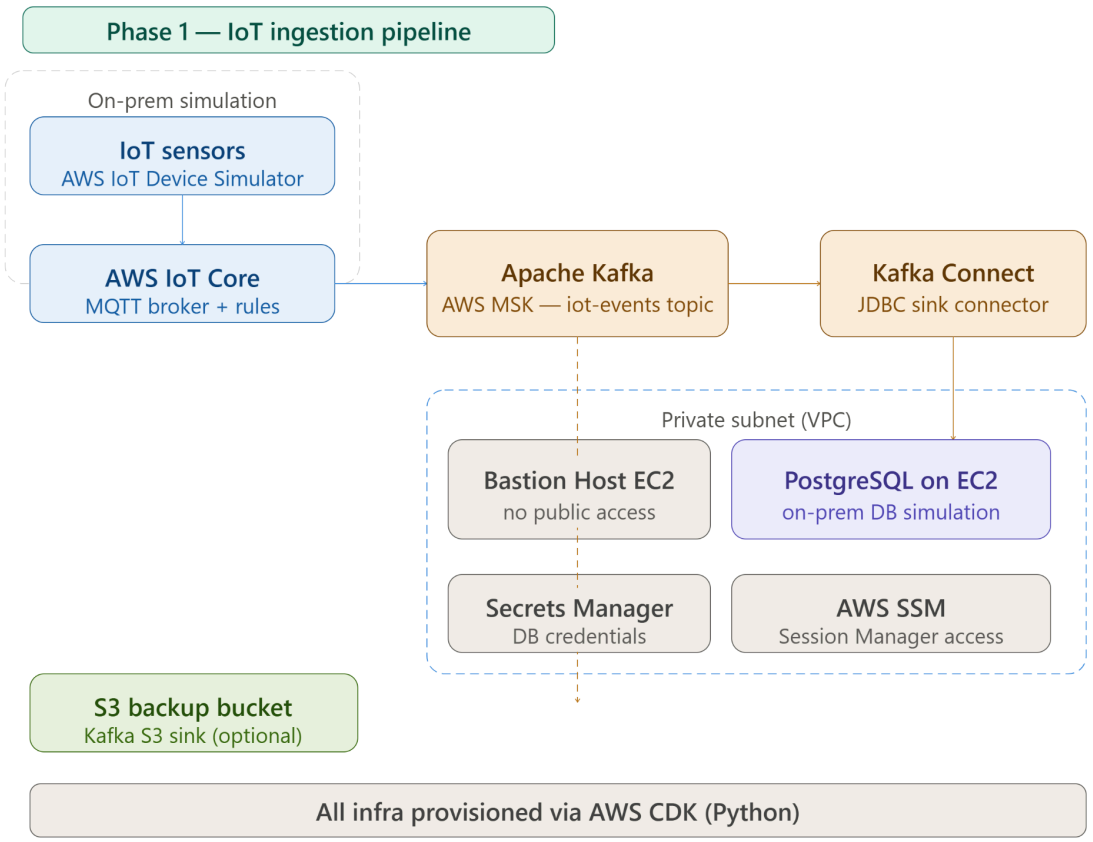
DATA ENGINEERING HACKATHON

On-Premise IoT Data Migration to AWS Cloud

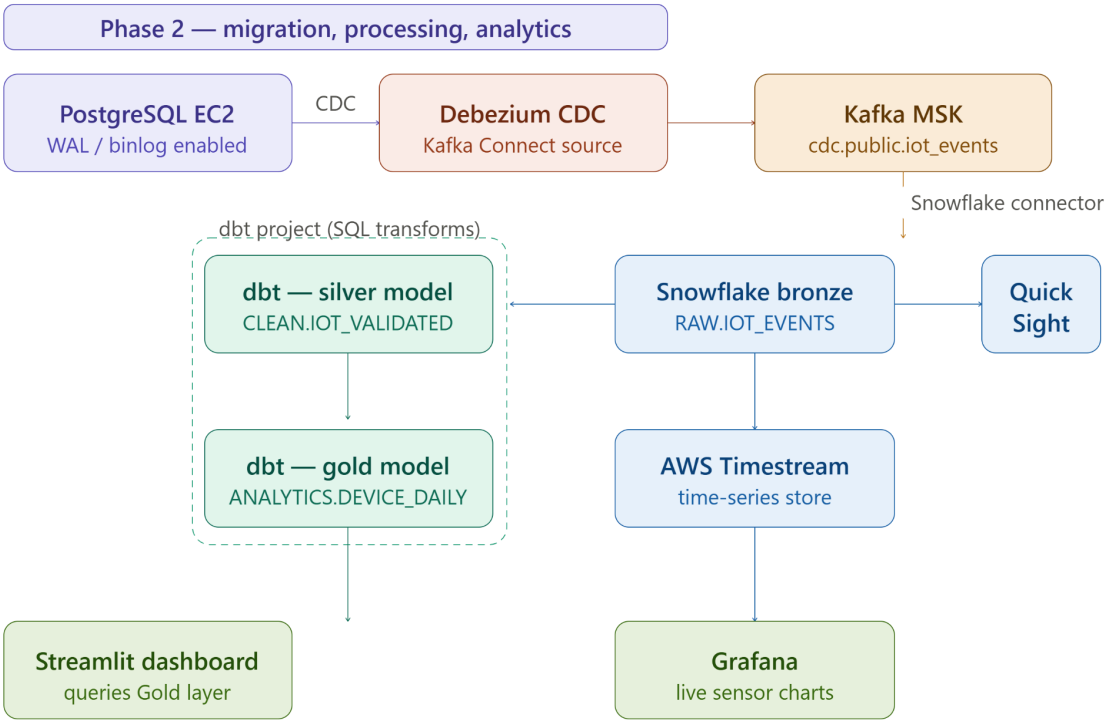
Project Overview

This hackathon simulates a real-world enterprise scenario: a company runs IoT sensors in an on-premise data center. The sensors generate continuous geolocation and telemetry data stored in a PostgreSQL database. Your challenge is to design and implement a complete AWS-based data pipeline that ingests this IoT data, migrates it to the cloud, processes it, and serves it through analytics and reporting tools.

PHASE 1



PHASE 2



Architecture Summary

Data Source	IoT Sensors → AWS IoT Device Simulator → MQTT → AWS IoT Core
Ingestion	Kafka MSK (iot-events topic) → Kafka Connect JDBC Sink → PostgreSQL EC2
Backup	Kafka S3 Sink Connector → Amazon S3 Backup Bucket
Migration	Debezium CDC → Kafka MSK → Snowflake Kafka Connector → Snowflake Bronze
Processing	dbt (SQL models) → Snowflake Silver → Snowflake Gold
Analytics	Snowflake Gold → Streamlit Dashboard
Time-series	AWS Timestream → Grafana Visualization

PHASE 1 — IoT Ingestion & On-Premise Simulation

Phase 1: IoT Data Ingestion Pipeline

In Phase 1, you will simulate an on-premise data center using AWS services. IoT devices are replicated using the AWS IoT Device Simulator, and their real-time data is ingested through AWS IoT Core into Apache Kafka on AWS MSK. Kafka Connect uses a JDBC Sink Connector to write events directly into PostgreSQL running on EC2 — simulating an on-premise server. A Kafka S3 Sink Connector provides the optional backup path.

Phase 1 — Architecture Flow

IoT Sensors (Simulated) → AWS IoT Core (MQTT) → Kafka MSK (iot-events topic) → Kafka Connect JDBC Sink → PostgreSQL EC2 + Kafka S3 Sink → S3 Backup

Phase 1 — Tech Stack

Component	AWS Service / Tool
IoT Simulation	AWS IoT Device Simulator (geoLocation data, O2 Arena London)
Message Protocol	MQTT via AWS IoT Core
Stream Delivery	Apache Kafka — AWS MSK (replaces Kinesis Firehose)
DB Writer	Kafka Connect — JDBC Sink Connector (replaces Lambda MySQL writer)
On-Prem DB	PostgreSQL on EC2 — private subnet (replaces MySQL)
Security	AWS Secrets Manager (DB credentials), VPC, Bastion Host
IaC	AWS CDK (Python) — provisions MSK, Kafka Connect, S3, VPC
Backup	Kafka S3 Sink Connector → Amazon S3 backup bucket
Access	AWS Systems Manager Session Manager (no public SSH)

Phase 1 — Tasks

Task 1.1 — AWS Account & Environment Setup

- Create AWS account and apply best practices (MFA, billing alerts, IAM roles)
- Install and configure AWS CLI with programmatic access
- Set up AWS CDK (Python) and install all dependencies
- Create AWS MSK (Kafka) cluster — note the bootstrap broker URLs

Task 1.2 — IoT Device Simulator

- Deploy the AWS IoT Device Simulator
- Create a device template for geoLocation data (latitude, longitude, timestamp, device_id)
- Start simulation of 5+ virtual devices in parallel
- Create an IoT Core Rule that routes MQTT messages to Kafka MSK topic iot-events

Task 1.3 — Kafka MSK & Kafka Connect

- Deploy Kafka Connect cluster connected to MSK
- Configure JDBC Sink Connector: topic = iot-events, target = PostgreSQL EC2
- Store PostgreSQL credentials securely in AWS Secrets Manager
- Optional: configure Kafka S3 Sink Connector for backup to S3

Task 1.4 — PostgreSQL EC2 (On-Prem Simulation)

- Launch an EC2 instance in a private subnet (no public IP)
- Install PostgreSQL and create the IoT schema and tables
- Enable WAL (Write-Ahead Log) — required for Debezium CDC in Phase 2
- Set up a Bastion Host EC2 for secure administrator access
- Use AWS SSM Session Manager to connect — no direct SSH key required

Task 1.5 — CDK Deployment

- Write CDK stacks for: MSK cluster, Kafka Connect, S3 bucket, VPC + subnet
- Deploy with cdk deploy and verify all resources created in Console
- Run the full simulation end-to-end and confirm data reaches PostgreSQL on EC2

PHASE 2 — Cloud Migration, Processing & Analytics

Phase 2: CDC Migration, Snowflake & dbt Analytics

In Phase 2, Debezium CDC captures every change from PostgreSQL in real time — replacing AWS DMS. Change events stream through Kafka MSK into Snowflake (replacing AWS RDS) via the Snowflake Kafka Connector. Inside Snowflake, dbt (replacing AWS Glue) runs SQL-based Bronze → Silver → Gold transforms. A Streamlit dashboard queries the Gold layer, and a parallel path writes to AWS Timestream for Grafana time-series visualization.

Phase 2 — Architecture Flow

PostgreSQL EC2 (WAL) → Debezium CDC (Kafka Connect source) → Kafka MSK (cdc.public.iot_events) → Snowflake Kafka Connector → Snowflake Bronze
Snowflake Bronze → dbt silver model → dbt gold model → Streamlit Dashboard
Snowflake → AWS Timestream → Grafana (time-series charts)

Phase 2 — Tech Stack

Component	AWS Service / Tool
CDC / Migration	Debezium CDC via Kafka Connect (replaces AWS DMS)
Event Transport	Kafka MSK — carries CDC change events to Snowflake
Data Warehouse	Snowflake — Bronze / Silver / Gold layers (replaces AWS RDS)
Transforms	dbt — SQL models for Silver and Gold layers (replaces AWS Glue)
BI Dashboard	Streamlit — queries Snowflake Gold layer
Time-Series DB	AWS Timestream
Visualization	Grafana (replaces QuickSight for time-series)
Language	Python, SQL, dbt (Jinja SQL)

Phase 2 — Tasks

Task 2.1 — Debezium CDC Setup

- Verify PostgreSQL WAL is enabled: wal_level = logical on EC2
- Deploy Debezium PostgreSQL connector inside Kafka Connect
- Configure: source = PostgreSQL EC2, output topic = cdc.public.iot_events
- Verify CDC events appear in Kafka topic (INSERT, UPDATE, DELETE events)

Task 2.2 — Snowflake Setup + Kafka Connector

- Create Snowflake account (free trial), database HACKATHON_IOT
- Create schemas: RAW (Bronze), CLEAN (Silver), ANALYTICS (Gold)
- Deploy Snowflake Kafka Connector — consume cdc.public.iot_events topic
- Verify Bronze table RAW.IOT_EVENTS is receiving CDC records

Task 2.3 — dbt Project

- Install dbt-snowflake, configure profiles.yml with Snowflake connection
- Write silver model: validate nulls, parse timestamps, add severity tags
- Write gold model: group by device_id + date, compute daily aggregates
- Run dbt run and dbt test — all models must pass
- Generate dbt docs and review the Bronze → Silver → Gold lineage graph

Task 2.4 — Streamlit Dashboard

- Connect Streamlit to Snowflake Gold layer using snowflake-connector-python
- Build 3 charts: device activity map, time-series AQI trend, top-N devices
- Dashboard must auto-refresh every 30 seconds

Task 2.5 — Timestream + Grafana (Bonus)

- Write a Lambda that reads from Snowflake and writes to AWS Timestream
- Connect Grafana to Timestream and build a live sensor panel

Submission Requirements

- GitHub repository with all code (CDK stacks, Kafka Connect config, dbt models, Streamlit app)
- Architecture diagram covering both phases (PNG or PDF)
- Screenshots of every deliverable listed above
- README.md with setup steps and how to run the full pipeline
- 5-minute demo: show live CDC event flowing from PostgreSQL to Snowflake Gold

Good luck — build it end to end!